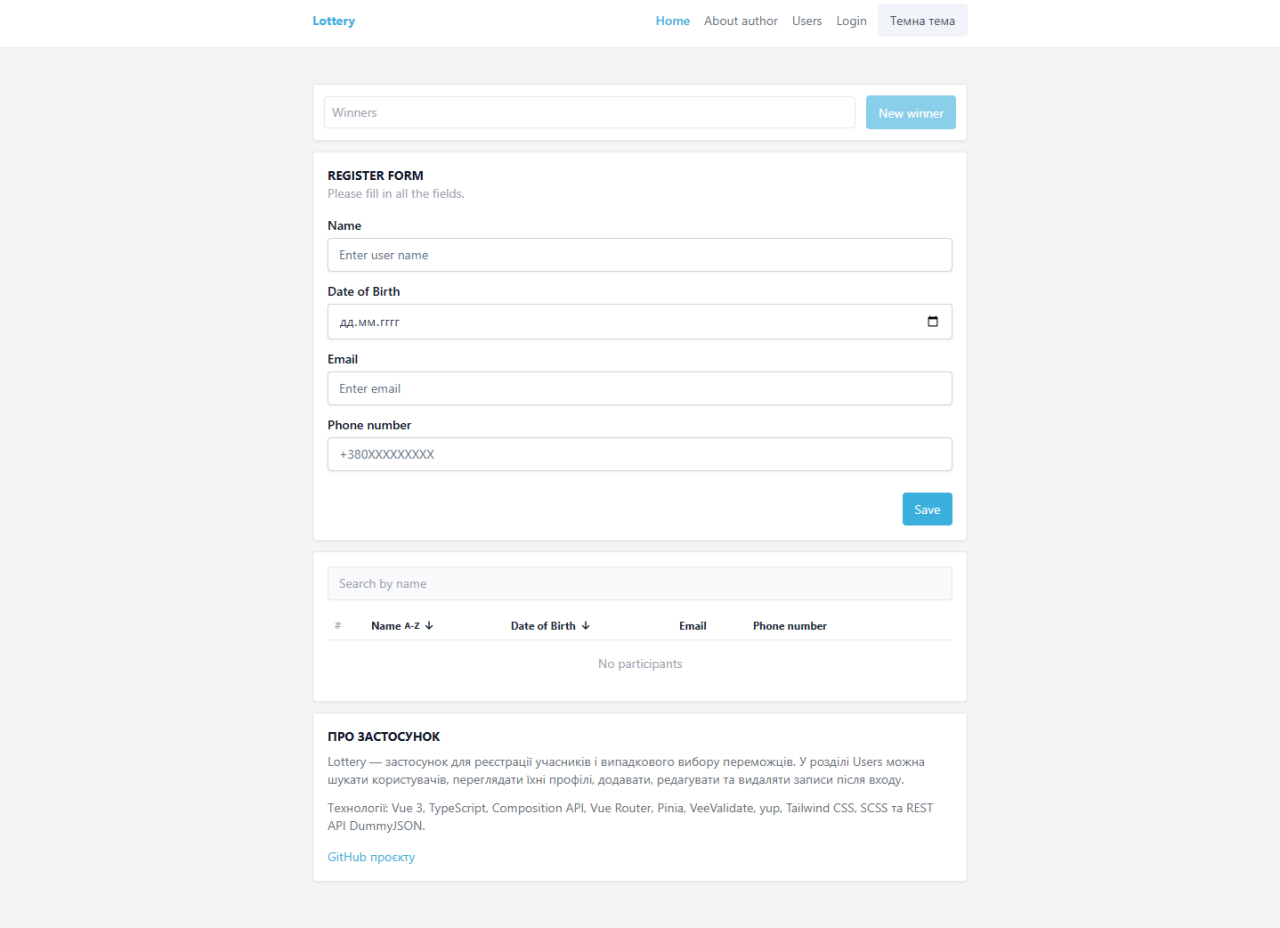
Vue Lottery
Архітектура FSD та Pinia

Лабораторна робота №6. Пояснення реалізації та підготовка до захисту.

Автор: Семенко Володимир Анатолійович
Репозиторій: Benswors/vue-lottery-app
Готова 5-та: feature/routing
Готова 6-та: feature/fsd
Vue 3 · TypeScript · Pinia · PrimeVue · VeeValidate · SCSS



Збережено лотерею, маршрути, Repository та JWT. У новій версії код організовано за FSD, додано обгортки PrimeVue, сповіщення, темну тему та автоматичний контроль імпортів.

Дата підготовки: 02.11.2026.

Як окремо показати 5-ту й 6-ту

Дві папки, дві гілки

5-та: C:\Users\ben\Documents\vue-lottery-app
Гілка: feature/routing
6-та: C:\Users\ben\Documents\vue-lottery-app-lab06
Гілка: feature/fsd

Не потрібно перемикати гілки в одній папці. Відкрий кожну папку в окремому вікні VS Code та виконай npm run dev. Звичайний порт 5-ї - 5173, порт 6-ї - 5176. Якщо порт зайнятий, використовуй адресу з термінала.

5-та: https://benswors.github.io/vue-lottery-app/
6-та: https://benswors.github.io/vue-lottery-app/lab-06/
Вхід: emilys / emilyspass

Нова копія з GitHub

```
git clone -b feature/fsd https://github.com/Benswors/vue-lottery-app.git
cd vue-lottery-app
npm ci
Copy-Item .env.example .env
npm run dev
```

У вже підготовленій папці залежності й .env є. Для локального запуску 6-ї адреса за замовчуванням: http://localhost:5176/vue-lottery-app/lab-06/. Файл .env містить VITE_API_URL та VITE_BASE_PATH і не входить у Git.

Команди перевірки

```
npm run lint
npm run lint:fsd
npm run test:fsd
npm run build
npm run test:e2e
```

FSD: шари, слайси й Public API

app → pages → widgets → features → entities → shared

app - main, App, роутер, guards, Pinia, тема, реєстрація сервісів і стилі.

pages - Home, About, Users, UserDetails, Login, 404 та сторінки створення/редагування.

widgets - AppHeader із навігацією та Logo, LotteryDashboard, UsersTable.

features - auth, user-form, participant-form, theme - дії користувача.

entities - user, session, lottery, settings - дані, типи, сервіси та stores.

shared - api, ui, lib, config - спільні технічні засоби.

Правила імпортів

Модуль може використовувати нижчі шари. Слайси одного шару не імпортують один одного. Кожен слайс експортує доступні частини через index.ts; зовнішній код не звертається до його внутрішніх файлів. Усередині одного слайса використовуються відносні імпорти. app та shared мають сегменти без предметних слайсів.

Дозволено: import { useUsersStore } from '@/entities/user'
Заборонено: import { useUsersStore } from '@/entities/user/model/users.store'
Заборонено: features/auth → widgets/app-header

Як довести, що лінтер працює

eslint.fsd.config.js налаштовує eslint-plugin-boundaries. Перевіряються напрямки та fileInternalPath=index.ts, включно з type та dynamic imports. test:fsd створює сім віртуальних порушень, перевіряє їх відхилення й додатково перевіряє дозволений імпорт. Навмисні помилки не додаються до вихідного коду.

Repository, Pinia та збереження

Repository у нових шарах

shared/api містить HTTP-клієнт, generic-контракти, Tokens та ServiceProvider. User і DTO - в entities/user/model, UserService - в entities/user/api. Конкретні сервіси реєструє app/providers/services.ts. Реєстр не імпортує entities, тому shared не залежить від верхніх шарів.

```
ServiceProvider.register(USER_SERVICE, () => new UserService())
ServiceProvider.register(AUTH_SERVICE, () => new AuthService())
```

Ключі сервісів типізовані. SessionUser визначено всередині session: слайс session не імпортує сусідній user. Сторінки й віджети використовують stores, а ті звертаються до сервісів через реєстр.

Setup stores

user store має users, total, isLoading, error, hasUsers та дії fetchUsers, fetchUser, createUser, updateUser, removeUser. session store має токени, профіль, isAuthenticated, setSession і clearSession. Початкову лотерею теж перенесено в Pinia. Для стану й getters у компонентах використано storeToRefs; actions можна отримувати напряму.

```
const store = useUsersStore()
const { users, total, isLoading } = storeToRefs(store)
```

Ручне збереження проти плагіна

Сесія вручну читається й записується в lottery:auth. Код явно перевіряє збережені поля та очищає токени після невдалого refresh. Потрібно стежити, щоб кожна відповідна action викликала persist.

Тема у settings використовує pinia-plugin-persistedstate: persist із key=lottery:settings та pick=[theme]. Плагін сам відновлює й записує дані. Менше шаблонного коду, але потрібно обмежувати поля для persistence. На session другий механізм не накладається: обидва підходи показані на різних stores.

PrimeVue та незмінений функціонал

Обгортки у shared/ui

AppButton використовує PrimeVue Button; **AppInput** - InputText із label і validation error; **AppTable** - DataTable/Column; **AppConfirmDialog** - підтвердження через Dialog; **AppNotifications** - Message для результатів операцій. Форми входу та користувача зв’язані з AppInput через VeeValidate.

Компоненти бібліотеки імпортуються лише в shared/ui та app/providers/ui.ts. Решта коду залежить від власних обгорток. Це зменшує обсяг змін при заміні бібліотеки. AppTable завантажується асинхронно; імпортуються окремі потрібні компоненти, а не вся бібліотека.

Тема та оформлення

Aura налаштована з основним кольором #3aafdc. Збережено блакитні кнопки, компактні поля й білі блоки. Перемикач змінює settings.theme; app синхронізує клас dark кореневого елемента, а PrimeVue використовує той самий селектор темної теми.

Що лишилося від 5-ї

Home із локальною реєстрацією та переможцями; Users із DummyJSON; пошук і сторінка в query; динамічний профіль і вкладене редагування; guards, redirect після Login, Logout, 404, title та layouts. Bearer і автоматичний refresh після 401 залишилися в HTTP-клієнті.

DummyJSON не записує дані

У store зберігаються created, changed та deleted. Вони накладаються на відповідь API перед локальним пошуком і пагінацією після мутацій. Новим користувачам надаються від’ємні локальні id. Зміни Users живуть до перезавантаження вкладки, а початкова лотерея Home зберігається окремо в localStorage.

Сценарій захисту на 5 хвилин

1. Дві версії (30 с). Відкрий 5-ту й 6-ту в різних вкладках. «feature/fsd створено від feature/routing. Попередня робота залишилася окремо; нова змінює архітектуру та додає UI-бібліотеку».

2. FSD (60 с). У VS Code покажи шість шарів, потім entities/user/index.ts. Поясни напрям імпортів. Запусти lint:fsd та test:fsd; покажи, що порушення перевіряються автоматично.

3. Repository (45 с). Покажи shared/api/service-provider.ts і app/providers/services.ts. «У shared немає залежності від UserService. Конкретні сервіси реєструються на рівні app».

4. Pinia (60 с). Покажи users.store.ts: state/getter/actions. У UsersTable знайди storeToRefs. Відкрий session.store.ts та поясни ручне збереження токенів і refresh.

5. UI та CRUD (60 с). Увійди, відкрий Users, створи користувача, відредагуй і видали. Покажи сповіщення та підтвердження. У shared/ui знайди обгортку AppInput і зв’язок із VeeValidate.

6. Тема й persistence (30 с). Увімкни темну тему й онови сторінку. Покажи settings.store.ts та persist. Порівняй з ручною реалізацією сесії.

7. Підсумкова перевірка (30 с). Покажи пошук у query, профіль /users/:id, вкладене edit і Logout. «Функціонал попередньої роботи збережено, зміни перевірено тестами».

Коротка розповідь

«Я розділив застосунок за відповідальністю. Сторінки складаються з віджетів і features, бізнес-стан живе в entities, технічний код - у shared. Зовнішні імпорти йдуть через Public API. Реєстр сервісів універсальний; stores керують API й локальними змінами. PrimeVue ізольована в shared/ui, тому сторінки не залежать від її конкретних компонентів».

Контрольні запитання / 1-8

1. Що таке FSD?

Методологія організації фронтенду за шарами й предметними слайсами. Вона задає місце коду та напрям залежностей, полегшує навігацію, розвиток і роботу команди.

2. Які шари й у якому порядку?

app → pages → widgets → features → entities → shared. Вищий шар описує ширший сценарій і може використовувати нижчі. Нижні шари не знають про верхні.

3. Головне правило імпортів?

Залежності йдуть вниз. Features не може імпортувати widgets. Усередині власного слайса дозволені відносні імпорти.

4. Чому не імпортувати сусідній слайс?

Це створює приховане зчеплення й цикли. Спільну технічну логіку виносять у shared, предметну - у нижчий відповідний шар. Взаємодію незалежних сутностей координує вищий шар.

5. Слайс і сегмент?

Слайс ділить код за предметною областю: entities/user. Сегмент ділить його за призначенням: model, api, ui, lib, config.

6. Public API та index.ts?

Контрольована точка доступу до слайса. Вона експортує дозволені компоненти, типи й stores. Інші слайси не залежать від внутрішнього розміщення файлів.

7. Entities проти features?

Entities описують сутності та їхні дані: user, session. Features описують дії користувача: вхід, форма створення/редагування, перемикання теми.

8. Чому app і shared без слайсів?

App містить інтеграцію всього застосунку, shared - спільну технічну основу. Їх організовують за сегментами, а не окремими предметними областями.

Контрольні запитання / 9-16

9. Чому старий ServiceProvider не підходить?

Він напряму імпортував UserService, а shared не може залежати від entities. Тепер shared має універсальний реєстр, а app реєструє конкретні сервіси.

10. Pinia проти composable зі спільним ref?

Pinia дає визначені stores, інтеграцію з Vue DevTools, getters/actions, плагіни та керування екземпляром стану через Pinia. Звичайний composable сам по собі цього не організовує.

11. Options store, setup store, storeToRefs?

Options store задає state/getters/actions об’єктом. Setup store повертає ref, computed і функції з setup-подібної функції. storeToRefs зберігає реактивність state/getters під час деструктуризації.

12. Чому не деструктуризувати state напряму?

Отримане значення може відірватися від реактивної властивості proxy. storeToRefs повертає пов’язані refs. Actions можна отримувати напряму.

13. Чому сервіс викликається через store?

Store централізує loading/error, локальні зміни та повторне використання даних. Компоненти зосереджуються на UI, а не дублюють роботу з API.

14. Навіщо UI-обгортки?

Вони ізолюють бібліотеку, уніфікують props і відображення помилок. Зміна бібліотеки насамперед зачіпає shared/ui.

15. Що таке tree-shaking?

Видалення невикористаного коду під час збірки. Потрібні окремі ESM-імпорти компонентів замість глобального підключення всієї бібліотеки. Важку таблицю також винесено в асинхронний компонент.

16. Плюси й мінуси FSD для лотереї?

Плюси: передбачувана структура, контроль залежностей, зручне розширення. Мінуси: більше файлів, Public API й налаштувань, вищий поріг входу. Для такого малого застосунку це насамперед навчання архітектури.